

INTRODUÇÃO A SISTEMAS OPERACIONAIS





Diretor Executivo

DAVID LIRA STEPHEN BARROS

Gerente Editorial

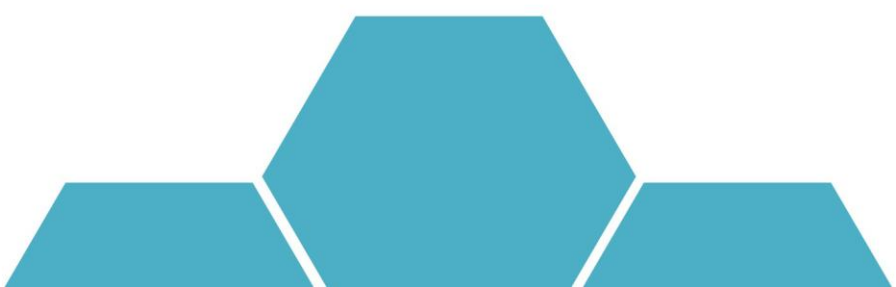
CRISTIANE SILVEIRA CESAR DE OLIVEIRA

Projeto Gráfico

TIAGO DA ROCHA

Autoria

JOÃO DANILO VENTURA HOLANDA NOGUEIRA



AUTORIA

João Danilo Ventura Holanda Nogueira

Sou formado em Ciência da Computação, com uma experiência técnico-educacional na área de Sistemas Operacionais de mais de dois anos. Lecionei na Faculdade Integrada da Grande Fortaleza. Sou apaixonado pelo que faço e adoro transmitir meu conhecimento àqueles que estão iniciando em suas carreiras. Por isso, fui convidado pela Editora Telesapiens a integrar seu elenco de autores independentes. Estou muito feliz em poder ajudar você, nesta fase de muito estudo e trabalho. Conte comigo!

ICONOGRAFICOS

Olá. Esses ícones irão aparecer em sua trilha de aprendizagem toda vez que:



OBJETIVO:
para o início do desenvolvimento de uma nova competência;



NOTA:
quando forem necessários observações ou complementações para o seu conhecimento;



EXPLICANDO MELHOR:
algo precisa ser melhor explicado ou detalhado;



SAIBA MAIS:
textos, referências bibliográficas e links para aprofundamento do seu conhecimento;



ACESSE:
se for preciso acessar um ou mais sites para fazer download, assistir vídeos, ler textos, ouvir podcast;



ATIVIDADES:
quando alguma atividade de autoaprendizagem for aplicada;



DEFINIÇÃO:
houver necessidade de se apresentar um novo conceito;



IMPORTANTE:
as observações escritas tiveram que ser priorizadas para você;



VOCÊ SABIA?
curiosidades e indagações lúdicas sobre o tema em estudo, se forem necessárias;



REFLITA:
se houver a necessidade de chamar a atenção sobre algo a ser refletido ou discutido sobre;



RESUMINDO:
quando for preciso se fazer um resumo acumulativo das últimas abordagens;



TESTANDO:
quando o desenvolvimento de uma competência for concluído e questões forem explicadas;

SUMÁRIO

Definição de sistemas operacionais.....	12
O que é um sistema operacional?	12
Sistema de processamento Batch	16
Sistema multitarefa.....	18
Sistema de computador pessoal	20
Sistemas com múltiplos processadores	21
Sistemas fortemente acoplados	23
Sistemas fracamente acoplados.....	24
Histórico dos sistemas operacionais.....	26
Como tudo começou?	26
Os computadores antes de Babbage	27
Os computadores depois de Babbage	28
Evolução do computador moderno	29
Década de 40.....	29
Década de 50	30
Década de 60	32
Década de 70.....	34
Década de 80	35
Década de 90	35
A virada do milênio	37
Década de 2010 - tempos atuais.....	38

Conceitos iniciais sobre sistemas operacionais	41
Hardware	41
Processador	42
Memórias	44
Dispositivos de entrada e saída	46
Barramentos	47
Arquiteturas de processadores	47
Software	49
Tradutor	50
Linker e Loader	50
Concorrência	51
Estrutura dos sistemas operacionais.....	54
Processos e endereçamento	54
Arquivos.....	55
Interpretador de comandos.....	57
Criando shellscripsts.....	59
Comunicação	61

UNIDADE

01

INTRODUÇÃO

Você sabia que chamamos de sistema operacional (SO), e na realidade, não é um sistema operacional?? Isso mesmo. Na verdade, o que hoje é chamado de SO é, apenas, um mecanismo de interação com o usuário que pode ser chamado de Shell, ou interpretador de comandos, quando temos um sistema com base em texto, ou uma GUI, nome dado para a famosa interface gráfica, usada na grande maioria dos computadores. Isso vale para diversos tipos de sistemas operacionais com os quais as pessoas, normalmente, têm contato como: Windows, Linux, iOS ou qualquer outro. Entendeu? Ao longo desta unidade letiva, você vai mergulhar neste universo!

OBJETIVOS

Olá. Seja muito bem-vindo à Unidade 1. Nosso objetivo é auxiliar você no desenvolvimento das seguintes competências profissionais até o término desta etapa de estudos:

1. Definir o conceito de sistemas operacionais.
2. Entender a evolução dos sistemas operacionais até o estado da arte.
3. Definir os conceitos iniciais relacionados aos sistemas operacionais.
4. Demonstrar a estrutura dos sistemas operacionais.

Definição de sistemas operacionais



OBJETIVO:

Ao término deste capítulo, você será capaz de entender como funcionam os sistemas operacionais. Entender o funcionamento de um SO é fundamental para qualquer um que deseja seguir na área da tecnologia, pois ele interfere diretamente em todo o processo tecnológico. E então? Motivado para desenvolver esta competência? Então vamos lá. Avante.

O que é um sistema operacional?

Muito se fala de sistemas operacionais e muito pouco se compreende sua real função. Um sistema operacional é um *software* que atua como um mediador de interação entre os usuários e os *hardwares* do computador. Ele é um *software* que operacionaliza a utilização de outros *softwares* para tornar as funcionalidades mais intuitivas.

Figura 1 - Sistema operacional



Fonte: Freepik (2021).

Para cumprir o objetivo de se tornar mais intuitivo, o SO conta com a utilização de dois mecanismos de interação: o Shell e a GUI. O Shell é um interpretador de comandos, um *software* que responde, via texto, para cada instrução que seja digitada em sua caixa. As GUI, *Graphical User Interface*, ou interface gráfica do usuário, são o mecanismo de interação mais comum nos computadores modernos de uso pessoal. Elas têm como base as imagens gráficas, cores e respostas visuais.

Vale a pena salientar que o uso de um mecanismo de interação não exclui o outro, pois os grandes sistemas operacionais de mercado, como diversas distribuições Linux (Debian, Ubuntu, Slackware, Fedora, Mint), as distribuições macOS e as distribuições Microsoft Windows, utilizam-se de uma combinação de Shell e GUI para sua total funcionalidade. Os SOs têm os chamados terminais de comando, sendo o Bash, em distribuições Linux e macOS e o CMD, em distribuições Microsoft Windows.

Mas tornando a dialogar sobre sistemas operacionais, propriamente ditos, trata-se de uma das partes mais importantes de um sistema computacional. Segundo Tanenbaum (1995), o sistema operacional pode ser chamado de **alocador de recursos**, sendo responsável por processar, escalonar e controlar os recursos de *hardware e software*, garantindo que eles possam se comunicar sem problemas.

Os SOs fazem parte do chamado sistema computacional, que pode ser dividido em quatro partes: *Hardware*, sistema operacional, *Softwares* aplicativos e usuários.

Figura 2 - Componentes de um sistema computacional



Fonte: Elaborado pelo autor (2021).

Segundo Tanenbaum (1995), os *hardwares* englobam toda a parte operacional física do computador, desde a unidade central de processamento, UCP, mais conhecida como CPU, as memórias e os dispositivos de entrada e saída de dados. Os *softwares* aplicativos

englobam a parte lógica, desenvolvida para cumprir funções específicas e facilitar a vida do usuário. Seus exemplos mais comuns são os aplicativos de leitura e edição de texto, executores de áudio, vídeo, navegadores da internet, aplicativos presentes em celulares, entre outros. Já os **usuários** são os atores que irão interagir com os *softwares* aplicativos para que eles realizem o processamento de dados. São aqueles que esperam os resultados dos processamentos e execuções das funções específicas dos *softwares* aplicativos. Podem ser tanto seres humanos como outros computadores. Por fim, o **sistema operacional**, que é o nosso foco principal, é o responsável por garantir o controle dos recursos de *hardware*, permitindo que todos os *softwares* aplicativos funcionem e apresentem seus resultados para o usuário.



REFLITA:

Para facilitar o entendimento, pense em um SO como o RH de uma empresa. Cada setor dessa empresa é um *software* diferente e o quadro de funcionários é o agrupamento de *hardwares*. O diretor da empresa é o usuário que precisa que cada setor apresente seus resultados. O RH da empresa irá alocar cada funcionário em setores e cada funcionário irá realizar sua função para gerar o resultado do setor. Por fim, esse resultado é enviado para o diretor da empresa que analisa todas as informações e pode aceitá-las ou solicitar uma nova demanda dos setores. Esse é o funcionamento de um sistema computacional.

Com base nessa análise, podemos inclusive perceber que a definição de sistema operacional pode, muitas vezes, ser confundida com a definição de sistema computacional. Isso acontece porque não existe uma definição, universalmente, aceita sobre o que é um sistema operacional. É mais fácil definirmos um SO pelo que ele *faz*, do que pelo que ele propriamente é. Tendo isso em mente, um sistema operacional é uma ferramenta que torna o uso do computador, conveniente para o usuário.

Tomamos, por exemplo, um sistema computacional comum, presente nas casas, empresas e ambientes educacionais. Esses sistemas têm diversos dispositivos de entrada e saída, monitores, mouses,

teclados, portas USB que podem receber *pen-drives*, leitoras de CD. Os mais antigos tinham *floppy* de disquetes. A forma de funcionamento desses dispositivos não é interessante para o usuário. Para ele, basta que os dispositivos funcionem, sem que se preocupe em porque ou como está ocorrendo.

Muitas vezes, ocorre de um usuário achar que uma interação de um dispositivo desses é simples, quando, na verdade, ela é composta por um conjunto de rotinas específicas que são, totalmente, manipuladas pelo sistema operacional, de modo que elas sejam acionadas de forma correta e eficaz. Em outras palavras, uma das funções do sistema operacional é ser um facilitador de acesso aos componentes desse sistema.

A grande maioria dos usuários acaba tendo essa visão de simplicidade por conta de definições rápidas dadas por algumas literaturas. Não vale a pena para, por exemplo, um usuário comum, entender todo o conceito que está em um SO, pois para ele, basta saber que o sistema operacional é o responsável pela interface gráfica, apresentada pelo computador. Esse conceito, embora incompleto, não está errado, é uma abstração de um todo para partes, também chamada de *visão top-down*.

Nós, da área da tecnologia, estamos mais acostumados com uma visão alternativa, que tem sempre o objetivo de abstrair as partes e relacioná-las a um todo, chamada de *visão bottom-up*. Enquanto a visão *top-down* tem o objetivo de simplificar conceitos, abstraindo suas partes, tornando fácil de compreender o funcionamento de uma determinada ferramenta, a *bottom-up* vem com o objetivo de listar todas as partes da ferramenta que contribuem para seu funcionamento.

Quando, por exemplo, o usuário de um SO executa três aplicativos simultaneamente, ele jamais irá imaginar que, no lugar daquele funcionamento, existe todo um processo de distribuição de processamento e que, na verdade, nada está sendo executado de forma simultânea.

Sistemas operacionais modernos podem realizar as chamadas “execuções simultâneas” de forma mascarada, quando, de fato, os processadores estão tão desenvolvidos que as trocas de processamento são imperceptíveis, de tão rápidas.

Esse é o conceito do compartilhamento de recursos do SO, no qual temos uma determinada lista de requisições e o sistema operacional irá tratar uma a uma, em uma fila de prioridade. Quando a requisição que tiver maior prioridade for executada por tempo suficiente, os recursos irão para a segunda requisição, em seguida para a terceira e assim, sucessivamente.

Exemplo: Pense no funcionamento de uma impressora, agora imagine que vários arquivos foram enviados para serem impressos nessa mesma impressora. Se não houvesse um sistema operacional, gerindo o funcionamento, poderíamos ter uma impressão em que partes da folha pertenceriam a vários arquivos diferentes. Porém, existe uma fila de impressão, que cuida para que os recursos físicos sejam utilizados para cada requisição, uma por vez, garantindo assim que, quando o processamento for concluído, o próximo seja iniciado tão logo, quando possível e, caso surja uma requisição prioritária, ela fure essa fila e seja executada, logo após o atual processamento.

Sistema de processamento Batch

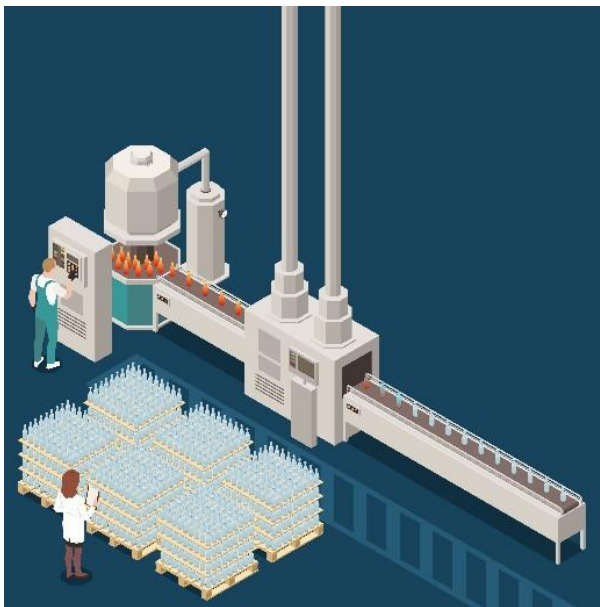
Antes dos computadores modernos existirem, todos os processos eram realizados de forma manual. Mesmo aqueles processos que já utilizavam alguma forma de automatização, por meio de computadores, precisavam ser inseridos manualmente, um a um, para seu processamento. Nessa época, os computadores eram máquinas fisicamente grandes, com toda a sua operação controlada em um painel que continha, geralmente, uma leitora de cartão ou fita e um dispositivo, que poderia ser uma impressora, um unidade de gravação de fita ou um perfurador de cartão, responsável por entregar os resultados dos processamentos de dados.

Como ainda não existia um dispositivo ou método de comunicação entre usuário e máquina, não havia interação direta do responsável pela operação com o sistema computacional. Para realizar os processamentos, o usuário preparava toda a tarefa, previamente, e a submetia para a execução. Dependendo da complexidade da tarefa, o processamento poderia levar um tempo extremamente longo, o que tornava a permanência do usuário na frente do console obsoleta. Os usuários submetiam as tarefas e iam fazer outras atividades, até que o resultado fosse emitido pelo computador.

Em algum momento, no entanto, se erros ocorressem ou se o processamento terminasse, antes do tempo estimado, a máquina ficava obsoleta, por só poder realizar uma tarefa por vez. Pensando nisso, os programadores da época começaram a organizar as tarefas em lotes de requisitos semelhantes.

O loteamento de processamentos e execução sequencial de tarefas de requisitos semelhantes denominam-se de processamento em lote, ou processamento Batch.

Figura 3 - Processamento sequencial



Fonte: Freepik (2021).

Nos sistemas computacionais antigos, mesmo o processamento Batch acabava deixando a máquina obsoleta, em alguns momentos, pois a diferença, entre a velocidade do processamento eletrônico e do processamento físico, era enorme.

Durante a execução das tarefas, uma determinada atividade precisaria executar uma ação de entrada ou saída. Enquanto essa ação ocorria, o sistema ficava ocioso, aguardando o momento em que poderia retomar o processamento e seguir com a execução da tarefa.

Com o avanço das tecnologias computacionais, surgiu um modelo de processamento chamado de multiprogramação. Os sistemas operacionais que se baseavam em multiprogramação tinham, uma certa autonomia, sobre o lote de processamentos que ele deveria realizar e, enquanto um determinado processo estava aguardando uma ação de entrada ou saída, o processador era realocado para outra tarefa, que era executada até que ela ficasse dependente de alguma ação física da máquina, passando, logo em seguida, para o próximo processamento, assim diminuindo o tempo de ociosidade do sistema computacional. Ao processo de decidir qual tarefa executar, enquanto outra tarefa esperava, dá-se o nome de **escalonamento de processos**.

Sistema multitarefa

O uso e o desenvolvimento de técnicas para o melhoramento do processamento Batch, incluindo o uso de multiprogramação, chegou ao ponto de evolução.

Segundo Machado (2007), o sistema multitarefa, de tempo compartilhado ou de compartilhamento de tempo, é um sistema que funciona com o princípio da multiprogramação. Diversos processos executados por vez, cada um ocupando o processador por um determinado período, porém, diferente do processamento Batch, em que os processos que ocupam o processador só são trocados, quando há uma dependência externa. No processamento multitarefa, as execuções entram e saem do processador, em intervalos de tempo fixos, permanecendo lá por tempo o bastante para executar uma instrução e, logo depois, é substituída por outro processo. Aqui, as trocas ocorrem com uma frequência tão acelerada que temos a impressão de presenciar o processamento simultâneo de tarefas.

Figura 4 - Multitarefa



Fonte: Freepik (2021).

Os sistemas interativos usam a técnica de processamento multitarefa para garantir que o usuário tenha sempre o melhor resultado possível da execução. Quando o usuário, por exemplo, alterna as janelas de programas, ele precisa ter a percepção de que tudo está ocorrendo de forma simultânea, na frente dele, porém, ali há todo o escalonamento de processos, definindo a fila de prioridade para os processamentos em lote. Essa técnica de processamento permite que múltiplos usuários usem os recursos da máquina, ao mesmo tempo.

Sistemas operacionais de tempo compartilhado têm uma complexidade maior do que um sistema operacional de multiprogramação. Embora ambos tenham similaridades, como o caso de terem a necessidade de manter várias tarefas na memória ao mesmo tempo, no caso do tempo compartilhado, a velocidade com que as tarefas são executadas exige uma rápida troca de alocação do processo, entre memória principal e disco.

Um dos métodos mais utilizados para garantir essa velocidade de forma eficiente, é a utilização das memórias virtuais. Esse método permite que algumas tarefas sejam executadas, sem que elas estejam, totalmente, na memória física da máquina, permitindo, assim, que o sistema computacional consiga executar, até mesmo tarefas que sejam maiores do que a própria memória física.

O uso da memória virtual elimina a limitação do tamanho da memória, fazendo com que programas, mais complexos, possam ser executados sem problemas.

Os sistemas de tempo compartilhado, geralmente, têm um sistema de arquivos que são armazenados em diversos discos, exigindo tecnologias de gerenciamento de arquivos e gerenciamento de disco.

Sistema de computador pessoal

Não é nenhum segredo que o desenvolvimento tecnológico não teve foco na experiência com o usuário, mas sim no poder de processamento e execução de tarefas. As máquinas eram criadas para serem capazes de realizar as mais diversas tarefas e, quando sofriam atualizações, eram aquelas que potencializavam os *hardwares* e a capacidade de processamento.

Com a chegada dos computadores pessoais, no entanto, uma nova linha de pensamento começou a ser aplicada nas linhas de produção. O usuário comum não tinha necessidade de um *hardware* potencializado, porque o usuário comum realizaria tarefas mais simples do que, por exemplo, o cálculo necessário para realizar um lançamento de um foguete espacial que irá aterrissar em outro planeta.

Surgiram então os modelos de sistemas operacionais, voltados exclusivamente, para computadores pessoais, especialidade de empresas como Microsoft e Apple e de comunidades de *software* livre que criavam distribuições Linux. Esses SO otimizavam a experiência do usuário, o tempo de resposta das execuções e o conforto da utilização dos terminais para a realização das mais diversas tarefas.

Coloca-se em questão, também, que computadores pessoais não tinham uma conexão de rede, inicialmente, então os sistemas operacionais não eram preparados para realizar a proteção de dados, porém, com o advento da internet e com a aplicação das máquinas em empresas, surgiu a necessidade do desenvolvimento de SOs capazes de realizar essa proteção de forma autônoma, embora existam diversos *softwares* de antivírus que prometem aprimorar essa proteção.

Figura 5 - Sistema de computador pessoal



Fonte: Freepik (2021).

Conforme a tecnologia aplicada nos computadores pessoais foi se aprimorando, *hardwares e softwares* mais poderosos começaram a ser utilizados, chegando em um momento tecnológico que dificulta a separação do que é a tecnologia do computador pessoal e da tecnologia, por exemplo, de *mainframes*.

Sistemas com múltiplos processadores

Com a necessidade de resultados mais rápidos, chegamos ao impedimento do nível de processamento das tecnologias atuais. Uma problemática que foi abonada com a aplicação da metodologia de criação de sistemas com múltiplos processadores.

Esses sistemas caracterizam-se por terem, duas ou mais unidades de processamento interligadas, trabalhando em conjunto para realizar as atividades de processamento.

Com a aplicação de sistemas com múltiplos processadores, as execuções começaram a ter seu tempo de resposta diminuído e criou-se a capacidade de termos mais de um processamento, ocorrendo simultaneamente. São computadores complexos, não voltados para o uso do público geral, com foco maior em desenvolvimento científico e militar, processamento de imagens e CAD.



VOCÊ SABIA?

Segundo Narayan (2008), CAD é o nome dado a *softwares*, utilizados principalmente por engenheiros, geólogos, geógrafos e *designers* para a criação e facilitação de projetos técnicos. São *softwares* que exigem um nível de processamento extremamente alto, devido ao detalhamento das informações que cada desenho deve passar, podendo ter diversas camadas que aumentam a exigência de processamento.

Um sistema com múltiplos processadores pode ser comparado a um sistema de multiprogramação, com um foco principal, na forma como é realizada o escalonamento das tarefas e a sua distribuição pelos processadores de forma que a tarefa seja executada, no menor tempo possível. Para falar desses sistemas, precisamos entender os conceitos relacionados às três vantagens de sua aplicação em atividades, que segundo Machado (2007) são: **escalabilidade**, **disponibilidade** e **balanceamento**.

Ainda, para Machado (2007), escalabilidade é a garantia que, a qualquer momento, eu posso ampliar a quantidade de processadores no sistema computacional, para potencializá-lo. Hoje, em um sistema computacional comum, caso haja algum problema em um processador, é necessário trocar todo o sistema por um novo, de forma a reconfigurar o funcionamento pelo novo dispositivo.

Disponibilidade é, talvez, a maior vantagem do uso de um sistema de múltiplos processadores. Imagine que você está na sua casa, trabalhando em um projeto para seu curso e, devido a uso constante, o processador do seu computador queime. No mesmo momento, o dispositivo iria desligar e só iria ligar novamente, se você trocasse o processador. Agora imagine que você não conseguiu o mesmo modelo de processador, esse trabalho seria ainda maior, pois todo o sistema computacional seria afetado e o sistema operacional precisaria ser trocado para se adaptar ao novo *hardware*.

De acordo com Machado (2007), a disponibilidade garante, por exemplo, que caso isso ocorra no sistema, ele continue disponível e funcional, realizando a distribuição dos processamentos que seriam feitos pelo processador que falhou, entre os demais processadores disponíveis.

Por fim, o balanceamento é a forma como os processos serão alocados, entre os processadores, de forma que eles sempre trabalhem similarmente balanceados (MACHADO, 2007), ou seja, se estou em um sistema de oito processadores e tenho 64 tarefas, cada processador irá, teoricamente, realizar oito tarefas cada, podendo, inclusive, serem tarefas de diferentes capacidades e diferentes exigências de processamento, algumas até serem feitas simultaneamente pelos oito.

Quando falamos de sistemas com multiprocessadores, falamos, principalmente de duas classificações: **sistemas fortemente acoplados** ou **sistemas paralelos** e **sistemas fracamente acoplados** ou **sistemas distribuídos**.

Sistemas fortemente acoplados

Um sistema fortemente acoplado é formado por conexões “fortes”. É um sistema computacional que tem um conjunto de processadores que compartilham, entre si, o mesmo conjunto de *hardwares*: memória física, dispositivos de entrada e dispositivos de saída. Também são sistemas que sofrem gerenciamento de um único sistema operacional.

Eles podem ser divididos em sistemas de multiprocessadores simétrico e sistemas de acesso não uniforme à memória. A diferença dos dois está no tempo de resposta aos processamentos. Enquanto o primeiro, tem um tempo simétrico e linear de acesso à memória principal, o segundo, por ser composto por diversos conjuntos de processadores acoplados, por meio de interconexões, tem um tempo de acesso não linear. Em sistemas paralelos, não existe um conceito de hierarquia de processamento, então, todos os processadores têm a mesma função.

Você vai perceber que toda tecnologia relacionada a computadores, que é desenvolvida, passa por um período de “larga escala de tamanho” e depois passa por um período de “encolhimento”. Com os sistemas

fortemente acoplados não foi diferente. Inicialmente, eles eram formados por dispositivos enormes, para caber os diversos processadores alinhados. Esses dispositivos eram utilizados, geralmente, por universidades e por grandes corporações que necessitavam de um processamento massivo e poderoso de dados.

Atualmente, com o avanço da tecnologia, os sistemas fortemente acoplados são utilizados na maioria dos sistemas operacionais modernos, principalmente naqueles com base em Unix e nos relacionados à família Microsoft.

Sistemas fracamente acoplados

Diferentemente dos sistemas fortemente acoplados, os sistemas fracamente acoplados não dividem uma única memória física, pois eles são formados por agrupamentos de sistemas computacionais, ou seja, cada integrante do sistema tem um sistema operacional próprio, uma memória física individual, seus próprios dispositivos de entrada e saída.

Uma curiosidade, é que a existência de um sistema fortemente acoplado, em um sistema fracamente acoplado, é possível, pois nada impede que um dos sistemas computacionais, listados no grupo, tenha mais de um processador.

Nesse modelo de sistema, os sistemas computacionais são interligados por redes de comunicação, podendo ser locais ou não. Os membros do sistema, por meio da rede, enviam requisições para o sistema computacional, definido como principal, e esperam as respostas para realizar seus processamentos.

Com o avanço das tecnologias de comunicação, surgiu um modelo novo de rede de computadores. Um modelo que difere dos anteriores, enquanto eles traziam um sistema computacional como o “host”, ou principal, o novo modelo trazia um conceito similar a “dividir para conquistar”, no qual diversos sistemas computacionais principais fazem parte desse sistema, descentralizando, assim, o acesso à informação.

Cada sistema computacional principal é responsável por parte das requisições, agilizando assim o andamento dos processos e acesso aos resultados.



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que um sistema operacional é o *software* responsável por gerenciar a comunicação de todas as partes de um sistema computacional e que, por isso, ele é considerado uma das partes mais importantes dele. Vimos também que existem muitas definições possíveis para o que seria realmente um sistema operacional, mas que é mais fácil definir um SO, por sua função e usabilidade, do que realmente por sua essência. Definimos então o conceito com o qual iremos trabalhar a partir de agora, o “alocador de recursos” que se utiliza de uma GUI ou Shell, para traduzir para o usuário, o funcionamento dos *hardwares e softwares* e, utilizando-se da mesma GUI ou Shell traduz os comandos e ações do usuário para esses *hardwares e softwares*. Vimos também os conceitos, envolvendo o compartilhamento de recursos, que é a funcionalidade principal de um sistema operacional. Foram abordados conceitos sobre tipos específicos de sistemas operacionais, salientando a diferença entre sistemas multiprogramados e multitarefados e de sistemas fracamente acoplados e fortemente acoplados.

Histórico dos sistemas operacionais



OBJETIVO:

Ao término deste capítulo, você será capaz de entender o avanço dos sistemas operacionais até o estado da arte. Faremos uma viagem por todo o túnel do tempo, entendendo os tipos de sistemas operacionais e suas usabilidades. E então? Motivado para desenvolver esta competência? Então vamos lá. Avante!

Como tudo começou?

É impossível falar sobre a história dos sistemas operacionais, sem falar sobre a história da evolução do computador como um todo. Sempre existiu o conceito de que o *software* apareceu para desafiar o *hardware* e esse evolui para suportar aquele. No dia a dia, podemos ver isso claramente, temos computadores com altas capacidades de processamento, memórias cada vez maiores, sistemas de armazenamento, cada vez mais rápidos e, sempre, acompanhados desse avanço, temos diversos *softwares* que exigem mais processamento, mais armazenamento e, cada vez mais, memória.

Para falar sobre a história dos sistemas operacionais, precisamos nos atentar a dois grandes marcos temporais. A máquina de Babbage, também chamada de máquina analítica ou *Analytical Engine*, considerada, hoje, como o mais próximo que temos de um computador moderno, criado, em 1833, por Charles Babbage, também chamado de “Pai do Computador”, juntamente com sua assistente Augusta Ada Byron, que ficou responsável por criar a sequência de instruções que eram executadas pela máquina. Ada Byron é considerada a primeira programadora da história.

Considerando Babbage, temos o período em que o computador se modernizou, juntamente com os sistemas operacionais, mas vale nos atermos à menção honrosa de todos aqueles que vieram antes.

Os computadores antes de Babbage

Um dos grandes nomes da programação moderna é Blaise Pascal, homenageado pela linguagem de programação Pascal. Mas o que muitos desconhecem é que Blaise Pascal foi um matemático francês que, em 1642, utilizando-se dos seus conhecimentos em matemática, inventou uma máquina de somar, a primeira do mundo, que ficou conhecida como Pascalina.

Figura 6 - Manutenção de engrenagens



Fonte: Freepik (2021).

Em 1673, quando a Pascalina só era utilizada por faculdades renomadas, devido ao seu alto valor comercial, Gottfried Wilhem Leibniz se dedicou a aperfeiçoá-la e, com seu sucesso, construiu uma máquina capaz de multiplicar e dividir.

As máquinas de Pascal e Leibniz foram utilizadas, constantemente, até o ano de 1820, quando outro matemático francês resolveu trabalhar em um invento capaz de realizar as quatro operações matemáticas. Charles Colmar então inventou o *Arithmomètre*. Uma máquina que teve mais sucesso comercial, pois suas antecessoras foram máquinas de sucesso acadêmico.

Em 1822, surge o divisor de águas. Charles Babbage trabalhou e lançou uma máquina para cálculos de equações polinomiais, a máquina diferencial. Mas foi, em 1833, que, com o algoritmo descoberto por Ada Byron, surgiu a ideia da máquina analítica.

Os computadores depois de Babbage

De acordo com Machado (2007), a máquina analítica nunca foi concluída por problemas de financiamentos, porém seu funcionamento e engenharia foi estudado e contribuiu para a criação do que hoje chamamos de computador moderno. E é por esse motivo que o título de Babbage se mantém como “Pai do Computador.”

Com o avanço nas engenharias das máquinas, foi necessário o estudo de técnicas de comunicação e lógica. Foi seguindo essa linha de estudo que George Boole chegou à definição da lógica Booleana, a base para o modelo de computação digital que é utilizada até hoje. Também chamada de lógica binária e lógica de Boole, foi um conceito usado no desenvolvimento de dispositivos com relés e válvulas, criando sistemas complexos de engenharia, capazes de ações automatizadas, nunca pensadas. Boole abriu as portas para o desenvolvimento dos primeiros computadores.

Em 1890, surgiu então a primeira máquina de cartões perfurados, criada por Herman Hollerith, para facilitar e acelerar os processos relacionados ao censo dos Estados Unidos. Hollerith, mais tarde, viria a fundar uma empresa que se tornaria a International Business Machine, IBM.

Enquanto a utilização de cartões perfurados era difundida, o ano de 1930 se tornou um ano de inovação. Na Alemanha surgia a Z1, a primeira calculadora eletrônica, que utilizava de relés e lógica binárias para chegar aos seus resultados. Nos Estados Unidos, surgia também a primeira máquina de cálculos lineares, a ABC (*Atanasoff-Berry Computer*), nomeado por seus criadores John Vicent Atanasoff e Clifford Berry. Essa é a máquina considerada o primeiro computador digital.

O ano era 1937, véspera do despoitar da Segunda Guerra Mundial, Alan Turing cria a ideia da máquina universal, a máquina de Turing, capaz de executar qualquer sequência de instruções de forma eficaz. Um modelo teórico que criou todo um novo pensamento e chegou a auxiliar na criação do modelo de **processamento de símbolos**, a base da ciência da computação. Turing também é chamado de “Pai da Computação.”

Evolução do computador moderno

Aqui, chegamos ao segundo grande marco temporal: A Segunda Guerra Mundial. Não é segredo que as guerras aceleram a pesquisa e o desenvolvimento de novas tecnologias, e foi a Segunda Guerra que acelerou o desenvolvimento do computador moderno. Mas porque isso é importante? Simples, os sistemas operacionais começaram a surgir, com o desenvolvimento dos computadores modernos.

Década de 40

Com o advento da Segunda Guerra Mundial, tornou-se, cada vez mais comum, encontrar máquinas de grandes dimensões, capazes de realizar os mais diversos tipos de cálculos. Essas máquinas podiam ser do tamanho de salas ou prédios inteiros. Nessa época, Alan Turing estava focado em desenvolver uma nova máquina, uma capaz de decifrar mensagens codificadas, principalmente, do exército nazista.

Foi na mesma época que a Marinha dos Estados Unidos começou a utilizar o Mark I, o invento de Howard Aiken, para realizar os cálculos de coordenadas náuticas. A máquina usava os mesmos princípios da máquina analítica, de Babbage.

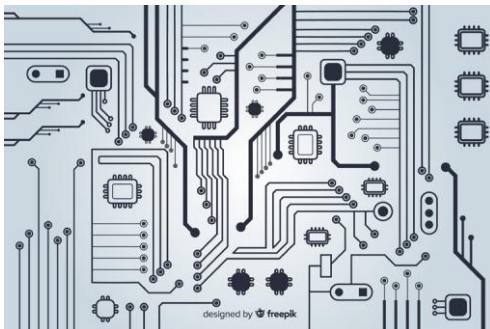
Com o fim da Segunda Guerra, os investimentos em tecnologia focaram em criar máquinas, capazes de maior processamento, desde o início da era criada por Pascal. Só em 1945, foi criado o primeiro computador eletrônico-digital. Ele recebeu o nome de *Electronic Numerical Integrator and Calculator*, o ENIAC.

Foi uma máquina revolucionária, capaz de realizar milhares de adições por minuto, seu principal uso conhecido foi o projeto da bomba de hidrogênio.

Por mais revolucionário que fosse o computador, programar o ENIAC requeria um conhecimento profundo do *hardware* da máquina. Sua programação consistia em painéis e conectores que deveriam ser posicionados, utilizando-se linguagem de máquina.

Embora a velocidade de processamento fosse excelente, o tempo necessário para codificar um programa começou a se tornar um empecilho para seu uso, o que levou a novas pesquisas. John Von Neumann, um dos consultores do projeto ENIAC, propôs o primeiro modelo de armazenamento de instruções e dados em uma memória.

Figura 7 - Circuitos eletrônicos



Fonte: Freepik (2021).

Com base no modelo desenvolvido por Von Neumann, criou-se o conceito de programa armazenado que é a base da arquitetura atual de computadores. Esse modelo, chamado de **arquitetura de Von Neumann**, pegou as ideias de Babbage (máquina analítica) e Turing (máquina de Turing) e unificou-as em um só projeto, finalmente colocando-as em prática. Deu-se início ao que foi chamado de **sistema computacional**.

Embora o conceito de sistema computacional começasse a ser aplicado no desenvolvimento de novos modelos de computadores, ainda não se trabalhava com mecanismos ou dispositivos, com função de interface com o usuário. Não existiam formas de entrada de dados como teclados, nem de saída de informação como monitores. As máquinas ainda eram utilizadas, unicamente, por universidades e órgãos militares. O conceito de sistema operacional ainda não havia sido aplicado.

Década de 50

Os computadores da época já eram mais usuais, mas não eram nada práticos. Seus tamanhos atrapalhavam processos simples, de manutenção. Foi, também, na década anterior que se popularizou o termo

“bug” para representar a existência de alguma falha no funcionamento. Insetos costumavam ficar presos entre as engrenagens e alavancas e causavam mau funcionamento.

Os físicos, do final da década de 40, inventaram um dispositivo capaz de amplificar e trocar sinais eletrônicos. Batizaram seu invento de transistor. Com seu tamanho diminuto, os transistores começaram a ser utilizados para substituir os grandes maquinários dos computadores.

Foi na década de 50 que a memória magnética começou a ser empregada. O uso combinado dos transistores e das memórias magnéticas permitiu um aumento na qualidade, confiabilidade e velocidade do processamento, além de rapidez e precisão no acesso de dados. Também foi ampliada a capacidade de armazenamento dos computadores e, conforme as engrenagens e alavancas foram sendo substituídos, começou-se também o processo de diminuição do tamanho das máquinas.

Com a diminuição do tamanho e barateamento da produção, grandes empresas começaram a atuar no mercado e a desenvolverem computadores para utilização em aplicações comerciais. O primeiro computador a atender esse objetivo de forma bem-sucedida foi o UNIVAC, *Universal Automatic Computer*.



VOCÊ SABIA?

A IBM nasceu em 1896, com o nome de Tabulating Machine Company e já naquela época dominava a tecnologia dos cartões perfurados. Só em 1924, que se tornou a International Business Machine, a empresa que continuou dominando a parcela de mercado por ser a única no mundo a dominar a tecnologia de seu lançamento. Foi ela que, no início da década de 60, lançou o primeiro computador eletrônico do mundo, RAMAC 305, que por sua vez, foi a primeira máquina comercial a utilizar um sistema operacional.

Programas específicos começaram a ser criados para as mais diversas áreas. Eles eram registrados em cartões perfurados que, quando submetidos a leitoras eram gravados em fitas. Os computadores eram

utilizados para realizar a leitura dessas fitas, executando as instruções gravadas, gerando uma fita de resultado. Quando um conjunto de programas é submetido a um computador, para sofrer esse tipo de processamento, chamados de **processamento Batch**.

Antes do surgimento do processamento Batch, os processadores ficavam ociosos entre uma execução e outra, porque as inserções de instruções eram feitas de forma manual e uma por vez. Para realizar automatização desse processo, alguns membros do Centro de Pesquisa da General Motors, utilizando-se do computador IBM 701, criaram o primeiro sistema operacional, registrado na história. Ele serviria de base para a criação de várias outras versões até a versão que chegou para lançamento comercial, juntamente com o RAMAC 305.

A década de 50 trouxe mais uma revolução nas tecnologias computacionais. Surgiram as primeiras linguagens de programação. Até aqui, sempre que alguma instrução era passada para o computador, a linguagem utilizada era a linguagem de máquina.

Com o surgimento das linguagens de alto nível, como COBOL, FORTRAN e ALGOL, os programas que antes eram desenvolvidos para serem executados em um determinado *hardware* e que, muito dificilmente, eram compatíveis com outros *hardwares*, passaram a ser mais “eletrônicos”, tornando a manutenção e evolução deles mais fácil e barata.

Os sistemas operacionais, seguindo a lógica da evolução das linguagens de programação, começaram a mudar para se tornarem mais simples, mais eficazes, para facilitarem codificação, execução e depuração dos aplicativos. Foi no final da década que surgiu o Atlas. O SO que introduziu a ideia de hierarquia de memória, criando os conceitos conhecidos e utilizados até hoje de memória principal, secundária e virtual.

Década de 60

A década de 60 trouxe um avanço no uso de transistores. Eles passaram a ser utilizados em circuitos integrados, o que fez o custo, que já era baixo, cair mais ainda, tornando o computador comercial mais acessível para ser utilizado em massa.

As grandes empresas então começaram a utilizá-los para compor seus escritórios. Com a procura aumentando, novas tecnologias de processamento foram desenvolvidas e os processadores começaram a ficar, cada vez mais, poderosos. As revoluções dessa década trouxeram diversas técnicas que usamos hoje para implementar sistemas operacionais, são elas: **multiprogramação**, **multiprocessamento**, **compartilhamento de tempo** e uma usabilidade mais encorpada da **memória virtual**.

Lembra-se do processamento Batch? A técnica de processamento em que permitia múltiplos programas serem executados de forma sequencial? Agora, surgiu o conceito de multiprogramação. Mas o que seria isso? É um modelo de processamento que permite que vários programas sejam executados na mesma memória. Enquanto no Batch, os programas eram enfileirados e executados, um por vez, aqui temos programas que executam parcialmente “simultaneamente”. Por exemplo, enquanto um processamento de leitura de dados ocorria, um processamento de escrita de dados poderia ser executado.

Nessa década, começou o processo de substituição das fitas por discos, tornando o uso dos processamentos Batch e a multiprogramação mais rápidos e eficientes. Vários programas eram carregados na memória e a gerência da fila de execução passou a ser dinâmica e não mais puramente sequencial.

Figura 8 - HardDisk (HD) - Disco magnético



Fonte: Pixabay (2021).

A análise e a depuração de programas, no entanto, ainda estava sendo um empecilho para os programadores da época. Erros de digitação e de lógica poderiam levar horas para serem encontrados. A tecnologia de compartilhamento de tempo foi lançada para que esse problema pudesse ser sanado, fazendo com que os SO garantissem que cada programa utilizaria o processador por pequenos intervalos de tempo.

O final da década de 60 foi marcado pela criação do mercado de minicomputadores, computadores de tamanho extremamente compacto, que poderiam ser mais facilmente comercializados. Foi também no final da década que Ken Thompson criou um sistema operacional que se popularizou e é utilizado como Kernel, base para vários sistemas operacionais modernos, o UNIX.

Década de 70

O advento dos anos 70 trouxe o conceito de microtecnologia. Tivemos aqui o lançamento do primeiro microprocessador, utilizado na composição do primeiro microcomputador. Esta década também foi marcada pela criação de duas linguagens de programação que, até hoje, são importantes para a área da Tecnologia da Informação: Pascal e C.

Os microcomputadores tornaram-se os computadores mais baratos já produzidos, o que induziu e aferventou o seu comércio, difundindo-os para vários usos possíveis. Foi nessa mesma década que Steve Jobs lançou o seu primeiro grande sucesso, o Apple II. Essa foi a década de fundação de duas das empresas que mais marcaram a evolução dos sistemas operacionais: a Apple e a Microsoft. Jobs e Gates competiram em tecnologia de forma tão intensa que forçaram a evolução da tecnologia, causando a criação de diversas arquiteturas de processamento e a principal delas foi a arquitetura de multiprocessadores.

A arquitetura de multiprocessadores levou um desafio aos programadores responsáveis por desenvolver os SOs. Eles precisavam de novos mecanismos de controle e sincronismos. Surgiu então o multiprocessamento, que permitia a execução de mais de um programa, agora, simultaneamente, ou até mesmo, quando necessário, a execução de um mesmo programa por mais de um processador.

Usando tecnologias de processamento vetorial e de paralelismo, agregadas ao multiprocessamento, foi lançado em 1976 o Cray-1, um microcomputador de processamento extremamente potente, composto por nada menos que duzentos mil circuitos integrados, capaz de realizar milhões de operações por segundo.

As GUIs começaram a se tornar, cada vez mais populares, e com elas os usuários comuns começaram a se mostrar interessados na usabilidade das máquinas.

As redes distribuídas, WAN (*Wide Area Network*), ficaram mais comuns, fazendo com que sistemas computacionais distintos passassem a conversar e compartilhar dados, independente da distância geográfica. As redes locais, LAN (*Local Area Network*), passaram a ser utilizados para interligar computadores e criar redes de acesso restrito. Com a popularização do uso de redes, os sistemas operacionais avançaram e se tornaram intimamente relacionados a elas.

Década de 80

A década de 80, trouxe o surgimento do computador pessoal, da estação de trabalho e dos sistemas operacionais, com base totalmente, em uma interface gráfica amigável ao usuário.

Embora as estações de trabalho fossem de uso de um usuário único, elas eram multitarefa, permitindo que diversas tarefas fossem realizadas de forma concorrente.

Um dos destaques dessa era foi, sem dúvida, o surgimento da tecnologia de protocolo TCP/IP, *Transmission Control Protocol/Internet Protocol*. Esse protocolo de rede é utilizado para uso da internet.

Década de 90

Os anos 1990 vieram com diversos avanços, relacionados a *hardware* e *software*. Podemos perceber que, durante a história, até esse momento, estavam sendo desenvolvidos *hardwares* mais potentes, mas pouco se falava da eficácia dos *softwares*. Eles eram criados para serem utilizados por aquele conjunto de *hardware*.

Isso mudou durante esta década. *Softwares*, cada vez mais potentes, foram surgindo o que obrigou a uma nova busca de formas de potencializar os *hardwares* para acompanhar esse desenvolvimento.

A evolução da microeletrônica possibilitou a então criação de processadores e memórias com maiores capacidade e de custos, extremamente inferiores. Em adendo, as mídias de entrada e saída sofreram uma redução de tamanho físico e uma ampliação de capacidade, ao mesmo tempo.

O Protocolo TCP/IP começou a ser mais frequente nos sistemas operacionais, devido à difusão do uso da internet. Com o aumento de usuários, surgiram os problemas de segurança e gestão do tráfego de dados. Os sistemas operacionais passaram a receber ferramentas que controlavam esse tráfego e procuravam garantir a segurança da informação. Também, se tornaram comuns os ataques de vírus ao computador.

Figura 9 - Rede de conexão



Fonte: Freepik (2021).

Foi a década de 90 que consolidou as interfaces gráficas dos sistemas operacionais. O lançamento das empresas Microsoft, Apple e Unix ganharam a simpatia dos usuários finais e os computadores pessoais de modelo *Desktop* começaram a entrar nas casas.

O Projeto Linux foi criado, em 1991, por Linus Torvalds, com auxílio e apoio de diversos programadores, criando uma comunidade que difundiu

o conceito de *software* aberto. Até o final dessa década, diversos produtos surgiram e foram disponibilizados, gratuitamente, na internet, atraindo diversos adeptos da filosofia.

A virada do milênio

Com computadores pessoais adentrando inúmeras casas pelo mundo e a internet se tornando um meio de entretenimento, além de uma forma de uma comunicação de dados, a virada do milênio trouxe um grande avanço nas tecnologias de processamento.

Os, até então já baratos, computadores se tornaram tão acessíveis que passaram de bem de luxo para bem comum. A eficácia e eficiência dos equipamentos trouxeram um novo viés comercial e começou-se a trabalhar em modelos portáteis.

Diversos portais de comunicação foram criados, periódicos que, antes, eram exclusivos em papel começaram a criar suas versões digitais. Comunicação pessoal, antes com base em telefonia e correios, começa a se digitalizar com a frequência do uso de *e-mails*.

Diversos modelos de portais, especializados em comunicação humana, começaram a serem lançados por todo o globo, fazendo surgirem assim as primeiras redes sociais: Orkut, Facebook, Twitter, Flickr, Google+. Surgem também os aplicativos de comunicação direta: ICQ, MSN, IRC.

O advento da internet gerou um novo modelo comercial, antes experimental. Nos anos 1990, já era comum existirem propagandas de aquisição de produtos por meio da TV e do rádio, mas com a internet, a ideia de comércio a distância popularizou-se e surgiram os primeiros *e-commerces*.

Os sistemas operacionais tornaram-se o grande foco do desenvolvimento da tecnologia. Agora eles estavam presentes nas vidas das pessoas por meio de computadores e, aos poucos, começaram a ser implantados em outros dispositivos, começando a ideia de ambientes integrados.

Os SOs começaram a ser implementados com técnicas de inteligência artificial, fazendo com que o tratamento de erros, prevenções de segurança e recuperação de dados ocorressem de forma automática, sempre que necessário. A área da Tecnologia da Informação começou a se tornar de extrema importância em todas as linhas socioeconômicas e as tecnologias começaram a mudar para suprir essas necessidades, criando arquiteturas mais robustas e sistemas distribuídos.

O final da década inicial do milênio, trouxe consigo uma nova visão do que era tecnologia. O que antes eram prédios e salas, agora estava disponível não apenas nas residências, como também espalhados nas mãos, com acesso facilitado com o toque. Surgiram os Smartphones e os sistemas operacionais embarcados.



DEFINIÇÃO:

Chamamos de sistema embarcado todo sistema operacional que é, completamente, dedicado a um dispositivo ou sistema que o controla. Os exemplos mais comuns os Smartphones. Um sistema operacional, totalmente, voltado para garantir o funcionamento de um dispositivo específico. Embora, por exemplo, possamos ter um Android Marshmallow em diversos dispositivos diferentes, na prática, o que existe é uma imagem do sistema operacional Android Marshmallow que foi adaptado e otimizado para o funcionamento em conjunto daquele agrupamento de *hardware*, ou seja, mesmo com sistemas operacionais de bases iguais, dois modelos diferentes de Smartphone teriam dois sistemas operacionais embarcados, totalmente diferentes.

Década de 2010 – tempos atuais

A ideia que até ali estava surgindo, aqui começou a ser popularizada. Com a facilidade de acesso e o baixo custo, os aparelhos *smartphones* se difundiram por todo o planeta. Dispositivos com capacidades de conexão à internet, que antes necessitava ser cabeada, agora utilizavam redes com base em conexões de ondas.

Com a comunicação de dados na palma das mãos e as diversas possibilidades de perdas de dados, novas tecnologias de armazenamento começaram a ser utilizadas.

Diversos serviços de armazenamento de dados surgem, utilizando-se da internet. O armazenamento em nuvem difunde-se, se tornando uma nova rama de negócios digitais. O armazenamento massivo em disco, que antes era um desejo e uma necessidade, agora passa a ser, cada vez mais, substituível por um armazenamento de nuvem.

A evolução da tecnologia móvel começou a desafiar os desenvolvedores de sistemas operacionais a criarem SOs mais robustos e com mais ferramentas e funcionalidades, voltadas à comunicação. As interfaces começaram a ser desenvolvidas para serem mais intuitivas e para garantirem uma compatibilidade na comunicação, entre os dispositivos móveis, *desktops* e servidores.

Figura 10 - Internet das Coisas (IoT)



Fonte: Freepik (2021).

Esse mesmo crescimento começou a trazer um novo conceito de usabilidade. Equipamentos que, antes eram puramente eletrônicos, começaram a se tornar digitais. Os sistemas operacionais foram aplicados em aparelhos de televisão, eletrodomésticos, equipamentos elétricos. Assim, surgiu a era da Internet das Coisas (*Internet of Things - IoT*).



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo tudinho? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que a criação dos sistemas operacionais está intimamente ligada ao desenvolvimento e evolução dos computadores. A evolução das máquinas foi lenta e gradual, usando as mesmas mecânicas e extremamente complexas. Diversos foram os nomes que tentaram criar automatizações de trabalhos manuais, mas os mais famosos foram Babbage e Turing. Enquanto Babbage teorizou uma máquina, capaz de realizar processamentos analíticos, Turing utilizou a teoria de Babbage para criar ideal de processamento, uma máquina capaz de executar, qualquer instrução que fosse inserida em seu processamento. Infelizmente, nenhum dos dois viveu o bastante para ver suas teorias aplicadas e utilizadas, no dia a dia e, inclusive, eles nem imaginavam onde suas teorias os levariam na história da computação. Chegamos então aos sistemas operacionais modernos. *Softwares* extremamente poderosos, capazes de processamentos impensáveis, disponíveis em aparelhos que cabem na palma das nossas mãos.

Conceitos iniciais sobre sistemas operacionais



OBJETIVO:

Ao término deste capítulo, você será capaz de entender conceitos relacionados aos sistemas operacionais. Para entender o funcionamento de um SO é preciso ter em mente diversos conceitos sobre técnicas e funcionalidades para os quais ele é preparado. Como visto anteriormente, temos sistemas computacionais de diversos tipos de formação, bem como sistemas operacionais, desenvolvidos para atender cada modelo daquele sistema. Entender o tipo de sistema computacional ou operacional com o qual se está tratando, é fundamental para garantir a melhor aplicação das novas implementações ou de manutenções. Mas, quais são os conceitos que vão auxiliar, diretamente, nos tratamentos com o sistema operacional? E então? Motivado para desenvolver esta competência? Então vamos lá. Avante!

Hardware

Muito falamos sobre *hardwares*, até o momento, mas o que realmente ele é? Para responder isso, vamos ter que lembrar que um sistema computacional é formado por um aglomerado de circuitos eletrônicos. Esses circuitos eletrônicos formam diferentes dispositivos que têm funções específicas no sistema computacional.

Existe uma numerosa lista de possibilidades da existência desses dispositivos em um sistema computacional, mas os mais comuns de serem encontrados são: **processadores, memórias, registradores, barramentos, monitores de vídeo, dispositivos de impressão, mouses, teclados e discos magnéticos.**

Segundo Machado (2007), a característica comum a todos esses dispositivos é a capacidade individual de manipular dados de forma digital, gerando resultados específicos de cada função de dispositivo, tornando-os úteis e confiáveis para a realização da transmissão de

dados por meio do próprio sistema computacional. Dispositivos físicos que realizam transmissão de dados e comunicação, nesse sistema, são chamados **hardwares**.

Figura 11 - Manutenção de *hardwares* por técnico



Fonte: Freepik (2021).

Os *hardwares* são divididos, no sistema, em três grandes agrupamentos: **unidade central de processamento**, composta unicamente pelo processador (veja na Figura 11); a **memória principal**, formada por um dispositivo de memória; e os **dispositivos de entrada e saída**. Esses três agrupamentos, ou unidades funcionais, estão presentes em todos os sistemas computacionais digitais, independente do modelo ou arquitetura. Algumas vezes, em formatos variados.

Processador

Todo sistema computacional é centralizado em dispositivos de *hardware* chamados de processadores. Os processadores, ou unidades centrais de processamento, são responsáveis por realizar todas as execuções de processar e o gerenciamento de ações de um sistema computacional, responsáveis, também, pelo controle de todas as unidades funcionais (outros *hardwares*), ligados ao sistema.

Os processadores são o cérebro da máquina. Todos os dados, em algum momento, passam por eles para que alguma ação seja executada. Eles são responsáveis por controlar e executar cada instrução, registrada na memória principal, por meio de operações lógicas, aritméticas ou apenas de gestão de dados.

Assim, existem unidades diferentes do processador que lidam com cada tipo de operação realizada por ele: uma unidade lógica e aritmética, uma unidade de controle e os registradores.

Cada unidade do processador tem suas funcionalidades bem definidas, por exemplo, a unidade de controle é a responsável pela execução de todas as atividades de cada componente do sistema computacional. É ela, por exemplo, que gerencia o processo de gravação e a leitura do disco magnético e o escalonamento de instruções, enfileirados na memória principal.

A unidade lógica e aritmética já descreve em seu nome sua funcionalidade. Cada operação lógica, sendo comparação e testes lógicos, e operação aritmética, somas e subtrações, são realizadas nessa parte do processador.

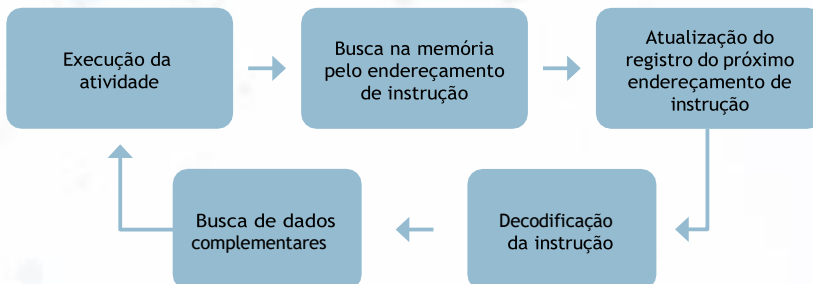
Por fim, temos os registradores. Eles são pequenos *hardwares* que compõem o *hardware* maior (processador). Eles, como o próprio nome já descreve, realizam o registro de dados. Para serem processados, os dados precisam estar em algum lugar de fácil acesso ao processador, então, durante uma execução, o processador registra esses dados, temporariamente, nos registradores e, quando o processamento é finalizado, ele emite o resultado e limpa dos dados registrados.

Os principais registradores que existem em um processador são específicos, com alguma funcionalidade, voltada ao funcionamento do sistema como um todo. Um processador tem um registrador que guarda o endereçamento de memória da próxima tarefa a ser executada, e que, toda vez que a tarefa é concluída, ele já se atualiza para apontar para o próximo endereço, por exemplo.

O funcionamento de um processador é um ciclo bem definido. De primeiro passo, o processador busca, na memória, o endereçamento registrado da próxima tarefa e então executa o processo de atualização do próximo endereçamento de memória. Ele então interpreta a instrução que recebeu, para a realização daquela tarefa e decodifica em linguagem de máquina. Caso existam dados complementares ou necessários na memória, ele realiza a busca e os carrega nos registradores. Quando os

registradores estão carregados com os dados necessários, finalmente, o processo é executado e então se inicia o ciclo novamente, como demonstrado na figura 12.

Figura 12 - Funcionamento de um processador



Fonte: Elaborado pelo autor (2021).

Memórias

As memórias são dispositivos que permitem ao sistema computacional a ação de armazenar dados, de forma permanente ou temporária. Os computadores modernos, geralmente, com base na arquitetura de Von Neumann, utilizam-se de memórias.

Memórias, quando realizam o armazenamento, organizam os dados em unidades de memória, a mais comum é o bit. Um bit é a menor unidade existente de memória, podendo assumir, apenas valores positivos ou negativos, aritmeticamente, descritos como **1** ou **0**, respectivamente.

Um sistema computacional trabalha com a existência de dois tipos de memória, as voláteis e as não voláteis. A volatilidade da memória está diretamente ligada à necessidade de energia para manter os dados, e as memórias voláteis são totalmente apagadas, quando a energia desaparece. Com as não voláteis, mesmo quando o sistema não está sendo alimentado com energia, os dados permanecem armazenados.

A memória principal, ou memória primária, é a memória onde os dados ficam armazenados, enquanto estão sendo utilizados. Ela é formada por células compostas por um determinado número de bits. É uma memória volátil. Na Figura 13, podemos ver o formato atual de uma memória principal.

Figura 13 - Memória principal



Fonte: Freepik (2021).

Para realizar o acesso ao conteúdo de uma célula da memória principal, o processador utiliza um registrador que ficou responsável por guardar um endereço de memória. Então, se determinada instrução quiser acessar informações na memória, ela precisa fornecer, ao processador, o endereço em que aquela informação foi armazenada.

As memórias secundárias são memórias não voláteis, responsáveis pelo armazenamento de modo permanente dos dados. São utilizadas para agrupar dados importantes que não, necessariamente, serão utilizados pelo processador imediatamente. Seus principais exemplos são as memórias de fita magnética, disco magnético e disco óptico.

Por fim, temos o terceiro tipo de memória, que também é uma memória volátil. A memória cache é uma memória de alta velocidade, utilizada para reduzir a disparidade entre as velocidades de execução das instruções pelo processador e a velocidade que os dados são gerenciados na memória principal.

Se colocarmos em ordem de velocidade de acesso, temos a memória secundária, com a mais lenta e a memória cache com a mais rápida. Em ordem de espaço de armazenamento, a velocidade de acesso é inversamente proporcional, sendo a memória cache a com menor capacidade de armazenar massividade de dados e a memória secundária, a com maior capacidade.

Dispositivos de entrada e saída

Um sistema computacional é formado por um sistema operacional, processador, *hardwares*, *softwares* e usuários, como já falamos. O objetivo geral da existência de um sistema computacional é a interação do usuário com os *hardwares* e *softwares* com o foco de determinar tarefas e receber resultados. Para isso ocorrer, são necessários dispositivos que permitam essa comunicação, transformando linguagem humana em linguagem de máquina e vice-versa.

A esses dispositivos, damos o nome de dispositivos de entrada e saída, utilizados pelo usuário, para comandar o sistema computacional (entrada) e para receber os resultados das execuções de suas tarefas (saída).

Os discos e fitas magnéticas são os dispositivos de entrada, utilizados pelo sistema computacional como memórias secundárias, geralmente, sendo acoplados ao sistema computacional e, raramente, são retirados de seus lugares.

Figura 14 - Dispositivos de entrada e saída mais comuns



Fonte: Freepik (2021).

Os dispositivos não necessariamente acoplados são variados, como teclados e mouses que têm objetivo de transmitir aos *hardwares* e *softwares* os comandos, definidos pelo usuário, de forma direta para gerar resultados. Resultados esses que podem ser exibidos, diretamente em telas, e monitores ou podem ser impressos ou plotados.

Barramentos

Barramentos são fundamentais no tratamento das unidades funcionais do sistema computacional. São os responsáveis por realizar a conexão de dados entre os dispositivos de *hardware*.

Seu funcionamento é simples. Eles têm dois registradores, um que registra a natureza da operação que está sendo executada, e outro que registra os dados que devem ser repassados de um *hardware* para outro. Os dados são endereçados e enviados para a memória principal, para assim passar pelo processador e a tarefa ser executada.

Agora, imagine que você está mexendo o mouse pela tela do seu computador. Enquanto você está movimentando o ponteiro, centenas de processamentos estão acontecendo e centenas de sinais estão sendo enviados do seu mouse, passando por barramentos, para só então entrarem na memória e serem processados pela unidade central de processamento. Agora perceba, também, que o ponteiro do mouse apresenta um movimento fluido e contínuo. Esse é o potencial de processamento dos computadores modernos.

Arquiteturas de processadores

Para falar sobre as arquiteturas, usadas nos processadores, é necessário entender um pouco sobre o conceito do que seria uma linguagem de máquina. Bom, quando falamos em computação, temos a linguagem humana, a linguagem de programação e a linguagem de máquina. A linguagem humana é a que nós utilizamos no dia a dia, a linguagem como os humanos costumam se comunicar.

Linguagem de programação é uma abstração da linguagem humana, criada para podermos nos comunicar com a máquina. O computador não entende a linguagem humana, assim como ele não entende a linguagem de programação, o que ocorre é que as linguagens de programação são criadas com base na arquitetura de um tradutor de código. Esse tradutor é o responsável por transformar a linguagem humana em uma linguagem que o sistema computacional possa compreender.

Existem linguagens de programação de alto nível, que são mais próximas da linguagem humana e são mais facilmente entendidas em uma leitura superficial, e existem as linguagens de programação de baixo nível, que são mais próximas da linguagem de máquina, a linguagem entendida pelo sistema computacional.

Cada processador vem com um conjunto de instruções, definidas pelo fabricante; são responsáveis por ditar a forma como esse processador irá trabalhar com seus registradores, controle de endereçamento e os tipos de dados que podem ser interpretados.

Se um programa for criado, em linguagem de máquina, podemos executá-lo diretamente no processador, sem a necessidade de interpretá-lo. Também pensando na ideia de que cada processador tem a sua lista de instruções, podemos dizer que cada processador fala uma linguagem diferente. As instruções diretas que funcionam em um processador A não serão executadas pelo processador B.

Um processador de arquitetura RISC (*Reduced Instruction Set Computer*), por exemplo, é criado com uma quantidade diminuta de instruções de máquina. Geralmente, as instruções aqui não acessam a memória principal e podem ser executadas, diretamente, no processador, sem passar por meios externos. A arquitetura RISC tem execução de tarefas rápida e tem fácil implementação de *pipelinings*.



DEFINIÇÃO:

Nós vimos que existe um tipo de processamento de dados, chamado de processamento Batch, em que o sistema operacional aloca um lote de tarefas e faz com que elas sejam executadas de forma intercalada para diminuir o tempo ocioso do sistema. Um *pipelining* é um outro modelo de processamento em lote, no qual esse lote de tarefas é executado de forma que cada tarefa seja particionada em pequenas instruções e entrem uma a uma, sendo executadas como em uma linha de produção (TANENBAUM, 1995).

Uma arquitetura CISC (*Complex Instruction Set Computer*), por outro lado, apresenta um conjunto de instruções complexas que necessitam de programas para serem executadas. Ela se diferencia da RISC também pelo número de registradores, enquanto a RISC tem muitos, CISC é caracterizada por ter poucos. Esse número reduzido de registradores dificulta a implementação de *pipelinings*.

Software

Depois de trabalharmos a definição de *hardwares*, está na hora de falarmos sobre o outro nome, citado diversas vezes: o **software**.

O *hardware* nada mais é do que um conglomerado de peças eletrônicas que reagem a impulsos elétricos. Na prática, sem um *software* eles não têm aplicação direta. É necessário existir um conjunto de programas para servir de interface de comunicação e que realizem a gestão de fluxo de dados para que os *hardwares* possam então funcionar devidamente no sistema computacional.

Podemos dizer então que o *software* é o gerenciador lógico do sistema computacional e que ele pode enviar diversas tarefas e instruções para serem executadas no processador e, após a conclusão do processamento, recolher os resultados e organizá-los para melhor apresentar ao usuário que disparou seu início.

Podemos fixar três nomenclaturas que são utilizadas, na área da computação, para trabalharmos com *softwares*. Um *software* de **sistema operacional** é o próprio sistema da máquina, quem faz a gestão direta e indireta de todos os *hardwares* e demais *softwares* que compõem o sistema computacional.

Mas, além do sistema operacional, os computadores modernos vêm com um conjunto de *softwares* que têm funções complementares ao SO. Eles são chamados de *softwares* **utilitários** e realizam serviços de apoio ao SO. São eles que cuidam de correções de erros, depuração de dados, carregamentos e ligações.

Por fim, os **softwares aplicativos** são aqueles que foram criados por usuários ou empresas, para a realização de objetivos específicos ou realização de tarefas cotidianas.

Tradutor

Como falado anteriormente, existem diversas linguagens de programação de alto nível e baixo nível. Essas linguagens de programação não são compreendidas pelo sistema computacional e precisam passar por um processo de tradução.

Os primeiros programadores eram obrigados a saber linguagem de máquina, já que as instruções eram passadas para o processador por meio de painéis e fios. Porém, com o desenvolvimento das técnicas de processamento, linguagens foram introduzidas no universo computacional.



REFLITA:

Vamos contextualizar os sistemas em Batch, sem conceitos de multiprogramação. Nessa etapa de processamento, em que um código não pode ser executado porque vai ficar, dependente de um código externo que precisa ser preparado, antes de ser executado, o sistema computacional se tornava ocioso e o tempo permanecia sendo gasto. Nos computadores modernos, esse processo simplesmente é realocado para outra posição na fila de execução e um novo processamento pode ocupar seu espaço.

Um tradutor carrega a instrução escrita em linguagem de programação e converte toda a representação simbólica em linguagem de máquina para só então iniciar a execução do código fonte. Muitas vezes, os códigos não estão prontos para serem executados, mesmo após estarem traduzidos, isso ocorre devido a existência de dependências externas que precisam ser ligadas ao processo de execução.

Linker e Loader

Após o tradutor gerar o programa de código fonte, ou módulo-objeto, cabe ao *linker* a função de realizar ligações entre os códigos

fontes que têm dependência entre si, para que assim seja gerado um programa executável que, pode ser listado na memória principal, para ser executado pelo processador.

Linkers são os *softwares* utilitários responsáveis por resolver todas as referências simbólicas entre dois, ou mais, códigos fontes. Ele faz isso, utilizando-se de bibliotecas de módulos-objetos.

O *linker* é o responsável por garantir o funcionamento do endereçamento da tarefa, gerada na memória principal, utilizando-se do princípio da relocação, para que sempre que o objeto seja carregado na memória, ele possa ser acessado. Para isso, o *linker* chama o processo do *loader*, que faz o trabalho de relocar esse processo sempre que é recarregado na memória principal.

É o *loader* que cuida do carregamento da memória principal. Todos os processos que estão enfileirados para serem executados são gerenciados por ele.

Concorrência

Um sistema operacional é um gestor para o sistema computacional. Ele é responsável por organizar e escalonar os processos, que serão executados, de forma ordenada e balanceada.

Podemos então dizer que um sistema operacional é um conjunto de processos que serão executados, competindo por recursos, ordenadamente. Isso ocorre porque o processador executa diversas tarefas, simultaneamente, fazendo com que os recursos do sistema computacional tenham que ser concorridos em cada processo.

Já vimos o conceito relacionado a multiprogramação. Se existe algo com uma característica multiplicativa, significa que existe algo com característica individual, então, vamos entender também o conceito de monoprogramação.

Em sistemas monoprogramáveis, cada processo entra de forma solitária no processador e permanece lá até seu encerramento. Todos os recursos do sistema computacional permanecem exclusivos para esse

processo. É um tipo de processamento que não utiliza o potencial do processador e causa ociosidade no sistema.

Nos sistemas multiprogramáveis, temos exatamente o conceito contrário, em que diversos processos são armazenados, na memória principal, e são executados de forma concorrente, com os recursos do sistema. Enquanto um processo aguarda uma operação de entrada/saída, por exemplo, outro processo ocupa o processador.

O princípio da concorrência diz que, quando um determinado processo perde o lugar no processador e retorna para a memória principal e logo em seguida, quando for recuperar seu lugar no processamento, esse processo deve estar exatamente igual ao momento em que deixou o processador, ou seja, a concorrência de processos exige que seja, criada uma imagem estática do momento do processo, para que esse possa ser continuado do mesmo ponto em que parou.

Para a existência da concorrência, também devemos nos atentar à necessidade de existirem interrupções e exceções. Quando, por exemplo, algum evento causa uma mudança no fluxo de processamento ou ocorre alguma falha de processamento, chamamos de interrupção ou de exceção, dependendo da natureza do evento.

Para entender esses dois conceitos, vamos contextualizar um caso: existe um processo A, que iniciou o processamento e solicitou uma requisição de entrada. Entrada e Saída são processos lentos, logo, o sistema operacional escalonou o processo B para executar, enquanto A ainda aguarda. Quando o *hardware* de entrada emite um sinal, informando ao processador que a requisição de A foi atendida, um processo de interrupção é disparado e interrompe o processamento de B, retomando, em seguida, o processamento de A.

Em resumo, uma interrupção sempre será gerada por causa externa à execução. Quando uma interrupção ocorre, o sistema operacional aciona um sistema de tratamento de interrupção, para garantir que o estado lógico daquele processo permaneça o mesmo até ele retornar para o processamento.

Exceções são geradas, sempre, como resultados do processo de execução. Quando ocorrem falhas de processamento, como por exemplo um ciclo infinito (*loop* infinito), uma superlotação de memória (*overflow*), uma divisão por zero ou o retorno nulo de uma solicitação que não deveria retornar nulo.

Assim como quando ocorrem interrupções, quando ocorrem exceções, o sistema operacional tende a acionar uma rotina de tratamento de exceções, para evitar que a falha cause danos diretos a outros processos.



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo tudinho? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que um sistema computacional é formado por *hardwares* e *softwares* e que dependem um do outro para operar, senão não passam de um conjunto de circuitos que responde a impulsos elétricos. Você deve ter visto os tipos de *hardwares* que podem compor o sistema computacional, seus funcionamentos, diferenças, classificações e papéis específicos no sistema. Quando falamos de *softwares*, foram listados os conceitos e suas divisões, os tipos existentes, funcionalidades no sistema computacional e as funções dos utilitários de tradução, ligação e carregamento de código. Também, foi abordado o conceito de concorrência, a capacidade do sistema computacional em permitir que seus recursos sejam concorridos pelos processos, armazenados na memória principal. Vimos um pouco sobre o que seria uma interrupção e uma exceção e sobre o princípio da concorrência.

Estrutura dos sistemas operacionais



OBJETIVO:

Ao término deste capítulo, você será capaz de entender a estrutura de um sistema operacional. Compreender as estruturas de funcionamento de um sistema operacional nos mostra o caminho a seguir para a criação das ferramentas de sistema (*softwares* utilitários). E então? Motivado para desenvolver esta competência? Então vamos lá. Avante!

Para a compreender o funcionamento de um sistema operacional, precisamos entender que ele é um agrupamento de instruções que estão sendo executadas de forma lógica e ordenada. Para que essa lógica e ordenação funcionem, é necessário que toda a estrutura do sistema seja bem definida. Vamos dar uma olhada, mais profundamente, em cada parte dessa estrutura:

Processos e endereçamento

Muito falamos sobre processos, processamentos, processadores. O que realmente seria um processo? Podemos dizer que é um dos conceitos mais básicos e mais importantes para os sistemas operacionais.

Todo *software* em execução irá gerar um ou mais processos, então podemos relacionar, diretamente, que um processo é um *software* ou parte dele que é executado pelo sistema computacional. Um processo tem instruções detalhadas para a execução de sua tarefa central. Essas instruções podem ser simples ou até mesmo complexas o bastante para se tornarem um novo processo.

Cada um deles, quando é alocado na memória principal, recebe um **endereçamento de memória**. Esse endereçamento consiste em um registro de posição, na memória, no qual aquele processo estará armazenado. Aqui, podemos relembrar que a memória principal é composta, por células divididas, em agrupamentos de bytes. O endereçamento diz para o processador em qual célula e em que byte específico ele irá encontrar o início do processo que irá ser executado.

No espaço endereçado, estão presentes todo o código executável, os dados a serem processados e o conjunto de recursos que serão necessários para a conclusão da tarefa. As principais chamadas do SO, quando se trata do gerenciador de processos, estão relacionadas ao início e final do processo. Sempre que um processo é iniciado pelo sistema, ele dispara uma rotina que irá realizar o seu encerramento.

Em todo sistema computacional, necessariamente na memória principal, existem espaços destinados ao armazenamento de informações, sobre os processos registrados, que entrarão na fila de processamento. Esses espaçamentos são organizados e registrados de forma que um espaçamento seja único, impedindo assim que o funcionamento de um processo interfira em outro.

Em uma situação ideal, os processos são enxutos e cabem, individualmente, em um endereçamento de memória, porém, com os avanços tecnológicos, processamentos, cada vez mais poderosos, foram se tornando mais comuns. e os processos que utilizavam diversos espaços de memória começaram a fazer parte do dia a dia do sistema computacional, o que acabava por diminuir a capacidade de armazenamento de um número maior de processos. Foi então aplicado o uso de memórias virtuais, uma técnica que separa parte do disco de armazenamento (memória secundária) para servir de apoio à memória principal, na qual o processo é inicialmente endereçado a essa memória principal e, complementarmente, endereçado no disco. Conforme o processamento vai ocorrendo, os endereçamentos são trocados de uma para outra até que todo o processo tenha passado pelo processador.

Arquivos

Trabalhando com o conceito de memória secundária, ou memória de armazenamento, temos então os arquivos. Aglomerados de dados, logicamente, dispostos de forma a exibirem resultados ou informações, pertinentes ao usuário.

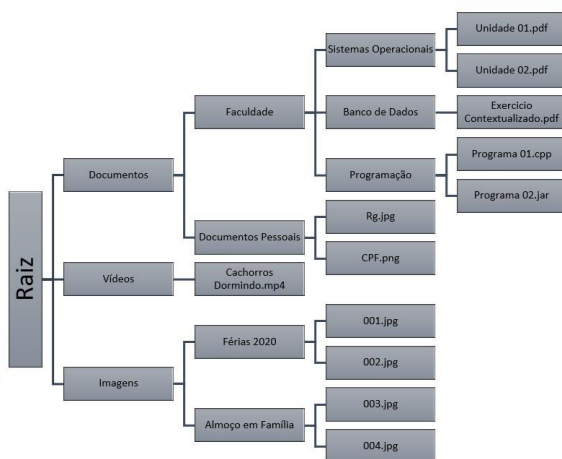
A base fundamental de todo o sistema operacional é o gerenciamento de arquivos, afinal, é necessário o registro dos arquivos,

na memória secundária, para serem executados com sucesso, durante o processamento de dados. Os arquivos são manipulados por meio de chamadas de sistema que podem mover, criar, editar, excluir ou copiar seus dados. A usabilidade de um arquivo é simples: ele pode ser aberto, lido e fechado.

Para trabalhar com arquivos, o sistema operacional fornece o suporte aos diretórios, que nada mais são do que agrupadores de arquivos. Você, por exemplo, pode ter um diretório com todos os arquivos referentes ao curso que está fazendo, agrupando-os de forma inteligente e que facilitem um acesso futuro. O sistema operacional oferece a capacidade de um diretório ter outros diretórios, o que cria a árvore de hierarquia de diretórios, na qual para você acessar um diretório filho, necessariamente, você tem que passar pelo diretório pai. A essa árvore hierárquica, chamamos **sistema de arquivos**.

Assim como na memória principal, os dados armazenados na memória secundária, também, são endereçados. Esse endereço é composto pela nomenclatura do disco, também chamada de **diretório-raiz**, em que se encontra o caminho para acessar o arquivo, passando por todos os diretórios necessários para chegar a ele. O resultado desse endereço é chamado de **caminho** ou **pathname**.

Figura 15 - Exemplo de hierarquia de arquivos



Fonte: Elaborado pelo autor (2021).

Com base no exemplo da Figura 15, podemos ver que o diretório-raiz tem três diretórios que contêm outros diretórios e arquivos. Para acessarmos, por exemplo, o arquivo “Unidade 01.pdf”, temos que acessar o seguinte Pathname: “/Raiz/Documentos/Faculdade/Sistemas Operacionais/Unidade 01.pdf”.

Interpretador de comandos

Antes de continuar, vamos falar um pouco sobre um SO específico. Para essa disciplina, iremos utilizar uma distribuição Linux, chamada de Ubuntu. Ela utiliza a base do sistema Debian.

Usaremos um sistema Linux, devido a característica de código aberto que nos permite maior liberdade para realizarmos operações das mais variadas naturezas. É comum a comunidade, inclusive, utilizar o sistema operacional como base e realizar centenas de modificações para torná-los únicos para cada usuário.

Figura 16 - Shell

```
Configuração de IP do Windows

Adaptador Ethernet Ethernet:

    Estado da mídia. . . . . : mídia desconectada
    Sufixo DNS específico de conexão. . . . . :

Adaptador de Rede sem Fio Conexão Local* 1:

    Estado da mídia. . . . . : mídia desconectada
    Sufixo DNS específico de conexão. . . . . :

Adaptador de Rede sem Fio Conexão Local* 2:

    Estado da mídia. . . . . : mídia desconectada
    Sufixo DNS específico de conexão. . . . . :

Adaptador Ethernet VMware Network Adapter VMnet1:

    Sufixo DNS específico de conexão. . . . . :
    Endereço IPv6 de link local . . . . . : fe80::2935:d6b8:2091:ce44%12
    Endereço IPv4 de Configuração Automática. . : 169.254.206.68
    Máscara de Sub-rede . . . . . : 255.255.0.0
    Gateway Padrão. . . . . :

Adaptador Ethernet VMware Network Adapter VMnet8:

    Sufixo DNS específico de conexão. . . . . :
    Endereço IPv6 de link local . . . . . : fe80::f408:daa0:f170:fdc9%8
    Endereço IPv4 de Configuração Automática. . : 169.254.252.217
    Máscara de Sub-rede . . . . . : 255.255.0.0
    Gateway Padrão. . . . . :
```

Fonte: Elaborado pelo autor (2021).

Dito isso, um sistema operacional é na verdade um código que realiza as chamadas de sistema, ou seja, o SO é um processo que chama outros processos, mas isso, nós já definimos anteriormente. Vale a pena lembrar que alguns sistemas operacionais trabalham com shell. O **shell** é o

interpretador de comandos mais comum, utilizado nas distribuições Linux e, recentemente, aderida por algumas versões do Microsoft Windows e também tem bastante similaridade com o interpretador de comandos do iOS, da Apple.

Cada linha digitada no shell gerará um subprocesso que irá executar e, quando encerrado, irá exibir o resultado de seu processamento. Vamos tomar como exemplo o Shell do Ubuntu, caso a instrução “date”, sem aspas, seja inserida, ele irá realizar o processamento e irá retornar a data atual do sistema operacional. Nós podemos, por exemplo, realizar combinações de comandos como, “echo ‘Olá Mundo’”, sem as aspas duplas. Esse comando vai fazer com que o interpretador de comandos, shell, ou terminal, exiba a saída “Olá Mundo”, porém, caso seja realizada uma modificação nesse comando e ele se torne “echo ‘Olá Mundo’>arquivo1”, também sem as aspas duplas, ele irá gerar um arquivo, no diretório em que está, e nesse arquivo irá escrever “Olá Mundo”.

Quadro 1 - Lista de comandos comuns ao interpretador de comandos do UBUNTU

Comando	Objetivo	Aplicação
Mkdir	Criação de um diretório	mkdir pasta01
Cp	Copiar arquivos ou diretórios	cp arquivo01 arquivo02
Mv	Mover arquivos ou diretórios	mv pathname1 pathname2
Cd	Acessa um determinado <i>Pathname</i>	cd pathname
rm	Remove um arquivo ou diretório	rm arquivo 01
Help	Exibe informações sobre os comandos	comando --help

Fonte: Elaborado pelo autor (2021).

O quadro 1 traz uma lista dos comandos mais comuns, utilizados pelo sistema LINUX. A lista de comandos do sistema operacional é infindável, cada um contém suas próprias funcionalidades e muitos são relacionados a softwares específicos. Na lista, temos instruções de gerenciamento do sistema de arquivos.

Criando shellscripts

Vimos que o Shell é o interpretador de comandos no Linux e que ele aceita a inserção de instruções diversas que realizam processos e retornam resultados.

Vimos, também, alguns comandos que são mais comuns no uso do *shell*, mas agora vamos realizar um procedimento de automatização desses comandos. Vamos criar um arquivo, chamado de *shellscript*, que será executado e irá realizar a sequência de instruções que nós definirmos, no interpretador.

Vamos começar com a preparação do ambiente. Já é comum as distribuições Linux virem com editores de texto instalados, mas vamos utilizar o editor de textos do próprio terminal do Linux. Caso ele não esteja instalado, basta executar o comando a seguir:

```
sudo apt-get update && sudo apt-get install vim
```

Sudo é um comando que invoca as permissões do administrador do sistema. Somente ele pode realizar algumas operações especiais ou instalações de pacotes de recursos.

Apt-get é uma chamada de instalador de pacotes de recursos; é um gerenciador que auxilia o sistema operacional a organizar os *softwares* que estão armazenados no disco. Ele pode ser combinado com comandos como “*update*”, para realizar a atualização de todos os pacotes instalados e “*install*” que irá instalar o pacote, cujo nome for listado, após o comando.

Então, essa linha de comando realiza a atualização de todos os pacotes de recurso instalados no seu sistema operacional e, logo em seguida, instala o *software* aplicativo “vim”.

Vamos criar um arquivo, com o vim, inserir uma sequência de comandos e então preparar a execução. Para criar o arquivo, podemos utilizar o comando “vim meu_shell.sh”, que irá criar um arquivo de texto com o nome “arquivo.sh”. Nesse arquivo iremos digitar:

Código 1 - Exemplo de Código ShellScript simples

```
#!/bin/bash
echo "Esse é o meu primeiro ShellScript"
numero_1 = 15
numero_2 = 16
soma = $numero_1 + $numero_2
echo "Um Shell Script é capaz de realizar operações matemáticas, quando
precisar"
echo "Por exemplo: $numero_1 + $numero_2 = $soma"
```

Podemos também criar condições e laços de repetição que executem um *shellscript*, criando execuções mais complexas e mais específicas. Podemos usar como exemplo o código abaixo:

Código 2 - Exemplo de código *shellscript* com condicional e laço de repetição

```
if ["$numero_1" > 10]; then
echo "O número é maior que 10"
fi
if ["$numero_2" > 20]; then
echo "o número é maior que 20"
elif ["$numero_2" < 10]; then
echo "o número é menor que 10"
else
echo "o número é $numero_2"
fi
for i in $(seq $numero_1)
do
echo $i
done
```

No código 2, temos dois modelos de condicional e um modelo de laço de repetição. No primeiro modelo, ele apenas irá informar, se o número é maior que 10. No segundo modelo, ele faz uma verificação para o número maior que 20 ou menor que 10, caso não seja nenhum dos dois, ele irá imprimir o próprio número. Por fim, o laço de repetição que irá fazer com que todos os números de 0 até 15 sejam impressos na tela, de 1 em 1.

Lembre-se, a implementação de um arquivo *shellscript* necessita sempre ser iniciada com a instrução “#!/bin/bash”, sem aspas. Retomemos o arquivo criado pelo vim, o “meu_shell.sh”. Ao concluir a inserção das instruções, linha a linha no arquivo, pressione a tecla ESC, no teclado e digite as teclas : e w, logo pressione **enter**. Essa ação irá salvar as alterações no arquivo. Repita o procedimento de pressionar ESC e agora realize a inserção das teclas : e q, logo depois pressione **enter**, novamente. O aplicativo vim será encerrado e nós retornaremos para o interpretador de comandos. Caso deseje, você também pode efetuar esse procedimento, utilizando **wq**, juntos, assim ele irá salvar e, em seguida, encerrar o aplicativo.

Agora vamos transformar nosso arquivo de texto em um *shell* executável, para isso, vamos mexer nas permissões gerais do arquivo. O comando que permite que nós alteremos as permissões de leitura e execução de um arquivo é o **chmod**.

```
chmod a+x meu_shell.sh
```

Ao executar esse comando, o arquivo se torna executável e passará a funcionar sempre que for chamado com a instrução “./meu_shell.sh”.

Comunicação

Sistemas operacionais modernos são preparados, para a realização de conexões entre máquinas, por meio de redes de comunicação. Existem três protocolos que são utilizados, de forma comum, para a realização dessa conexão e troca de pacotes de dados: TCP, SSH e FTP.

O protocolo TCP é o mais utilizado para realização de conexões de rede, geralmente, relacionado à rede WAN. Ele é que permite a conexão de um sistema computacional com a internet. Com ele, por meio de servidores e endereçamentos WEB, é possível realizar o envio e recebimento indireto de dados.

Protocolos SSH e FTP diferem do TCP no caráter de diretividade. Enquanto o TCP utiliza uma ferramenta para realizar o intermédio de comunicação, os protocolos SSH e FTP conectam dois sistemas computacionais, diretamente entre si, e permitem a interação de dados.

O Protocolo SSH, *Secure Shell*, garante a conexão entre o cliente (sistema computacional requisitante) e o cliente remoto (sistema computacional acessado). Ele permite a manipulação e comunicação de dados, bem como a transferência de arquivos.

Ele funciona de forma a criptografar os arquivos que estão transitando de um sistema para o outro, evitando assim que, caso eles sejam interceptados por um *software* malicioso, possam ser lidos por terceiros.

O FTP, *File Transfer Protocol*, é um protocolo de transferência de arquivos. Enquanto o SSH permite a manipulação do sistema computacional, acessado remotamente, o FTP permite apenas a comunicação e transferência de dados.

As distribuições Linux são variadas e implementadas para diversos tipos de objetivo, logo, algumas delas já podem vir com aplicativos que garantam ou facilitem a utilização de protocolos de transferência de arquivos.

O Ubuntu não está nessa lista de distribuições, logo, para utilizar o serviço, teremos que realizar a instalação dos pacotes necessários. No *shell* do seu sistema operacional, execute o seguinte comando:

```
sudo apt-get install openssh-server  
sudo service ssh start
```

Para se conectar com o sistema operacional, é necessário utilizar o comando `ssh`, da seguinte forma:

```
ssh nomeusuario@ip-remoto
```

O “nomeusuario” é o nome do usuário do computador remoto, “ip-remoto” é o endereçamento IP do computador remoto. É possível também acessar por meio de uma porta de conexão específica e ainda conectar, automaticamente, já passando na instrução a senha de conexão. Para usar outra porta, que não seja a padrão, basta adicionar no final da instrução o complemento “-p numeroporta”, sem aspas.

Para realizar a passagem automática da senha, precisamos utilizar um outro pacote do sistema operacional. Execute os comandos:

```
sudo apt-get install sshpass
```

```
sshpass -p “seu_password” ssh nomeusuario@ip-remoto
```

Para realizar a transferência de arquivos, utilize, no lugar de `ssh`, o comando `scp`, seguido do pathname que contém o arquivo:

```
scp nomeusuario@ip-remoto:/home/documentos/faculdade/sistemas_operacionais/unidade_01.pdf /home/documentos
```

Esse comando fará com que o documento “unidade01.pdf”, que está em outro sistema computacional, seja copiado para meu sistema computacional, nos diretórios “documentos.”



RESUMINDO:

E então? Gostou do que lhe mostramos? Aprendeu mesmo? Agora, só para termos certeza de que você realmente entendeu o tema de estudo deste capítulo, vamos resumir tudo o que vimos. Você deve ter aprendido que um sistema computacional tem uma estrutura bem definida, e que tudo funciona de forma organizada e objetiva. Os famosos processos, tão citados em outros capítulos, são a alma do sistema operacional e são movimentados o tempo inteiro entre as memórias principal, secundária e o processador. Viu também a importância do endereçamento de memória e entendeu o conceito do sistema de arquivos. O sistema de arquivos, como base do sistema de armazenamento da memória secundária, é organizado de forma hierárquica e pode ser acessado também por meio de um endereçamento de memória. Você entendeu o conceito do interpretador de comandos, descobriu alguns comandos comuns e suas funcionalidades e até mesmo montou seu primeiro *shellscript*. Agora você é capaz de criar *shellscripts* variados para atender os mais diversos propósitos no seu sistema computacional. Falamos também sobre comunicação de dados, focando em protocolos de transferência de arquivos. Realizou a primeira configuração e entendeu como funciona o protocolo SSH para manipulação de clientes remotos e transferência de arquivos.

REFERÊNCIAS

MACHADO, F. B.; MAIA, L. P. **Arquitetura de sistemas operacionais**. Rio de Janeiro: LTC, 2007.

NARAYAN, K. L. *Computer aided design and manufacturing*. New Delhi: Prentice Hall of India, 2008.

TANENBAUM, A. S. **Sistemas operacionais modernos**. São Paulo: Guanabara Koogan, 1995.

SILBERSCHATZ, A.; GALVIN, P.; GAGNE, G. **Fundamentos de sistemas operacionais**. Rio de Janeiro: LTC, 2010.

SILBERSCHATZ, A. GALVIN, P.; GAGNE, G. **Sistemas operacionais: conceitos e aplicações**. Rio de Janeiro: Campus, 2000.

